

Intermediate (feel the speed difference) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real caller would actually handle this). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

You're going to run the same 3 slow tasks two different ways, and time both, so you can see the difference instead of just being told about it.

First way: run task 1, wait for it to finish, then task 2, then task 3. That's "one after another" — the total time is the sum of all three. Second way: start all three at the same time and let them run together — the total time is roughly however long the *slowest* one takes, not the sum.

Things to use:

- `async def f1(): await asyncio.sleep(2)` — a slow task, times 3.
- `time.perf_counter()` — start a stopwatch, then read it again later.
- `await f1(); await f2(); await f3()` — one after another.
- `await asyncio.gather(f1(), f2(), f3())` — all together.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The exercise is really about two `asyncio` features working together: `await` running things one at a time, and `asyncio.gather` running them together, with `time.perf_counter()` measuring the actual difference.

`await asyncio.sleep(2)` pauses a task for 2 seconds without blocking anything else — that's the whole point of `async`. Calling `await f1(); await f2(); await f3()` runs them strictly in order — each `await` waits for the previous one to fully finish before starting the next. Calling `await asyncio.gather(f1(), f2(), f3())` starts all three immediately and waits for all of them together — so the total time is close to the single longest one, not the sum.

The exact pieces:

- `async def f1() -> str: await asyncio.sleep(2); return "f1 done"` — same shape for `f2`, `f3`.
- `time.perf_counter()` — a high-resolution clock meant exactly for timing short spans of code.
- `async def run_sequential() -> None:` — calls `await f1()`, then `f2()`, then `f3()`, timing the whole block.
- `async def run_parallel() -> None:` — calls `await asyncio.gather(f1(), f2(), f3())` once, timing that single call.
- `asyncio.run(...)` twice, separately — you can't call `asyncio.run` from inside another running event loop.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think about what happens once these 3 tasks aren't 3 identical `sleep(2)` calls, but 3 real API calls, and one of them is slow or genuinely broken. `asyncio.gather(f1(), f2(), f3())`, by default, doesn't fail as soon as one task raises — it keeps the others running, then re-raises the *first* exception it sees once everything has settled. That's usually fine. But it also means: if `f2` raises immediately and `f3` was about to make an expensive, unnecessary API call anyway, `gather` doesn't stop it from happening just because a sibling task already failed.

The real design question isn't just "how do I run 3 things at once" — it's "what should happen to the *other* two tasks if one of them fails or is still running when I no longer need the rest?"

The extra piece that answers that question:

- `return_exceptions=True` — `asyncio.gather(f1(), f2(), f3(), return_exceptions=True)` changes `gather`'s behavior: instead of raising the first exception immediately, it waits for every task to finish (or fail) and gives you back a list where a failed task's slot holds the exception object itself, not the result — so one bad call doesn't wipe out the 2 good results you already got. Without it, a single failure in one of three otherwise-successful calls loses you all 3 results, not just the broken one.

Sketch what your `run_parallel` output would look like with one of the 3 tasks raising, with and without `return_exceptions=True`, before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both assume all 3 tasks succeed — they only ever prove the speed difference. Advanced asks what happens the moment one of them doesn't, which is the realistic case once these aren't `sleep()` calls but real network requests — and shows the one flag (`return_exceptions=True`) that decides whether one failure costs you every result, or just that one.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make three slow functions f1, f2, f3:
    each one waits 2 seconds, then is done

make a function run_sequential:
    start the stopwatch
    wait for f1, then wait for f2, then wait for f3
    print how much time passed (should be about 6 seconds)

make a function run_parallel:
    start the stopwatch
    wait for f1, f2, and f3 all together, using gather
    print how much time passed (should be about 2 seconds)

run run_sequential
run run_parallel
compare the two printed times
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# gather_practice.py – Intermediate section
import asyncio
import time

async def f1():
    await asyncio.sleep(2)
    return "f1 done"

async def f2():
    await asyncio.sleep(2)
    return "f2 done"

async def f3():
    await asyncio.sleep(2)
    return "f3 done"

async def run_sequential():
```

```
start = time.perf_counter()
await f1()
await f2()
await f3()
print("sequential:", time.perf_counter() - start)
```

Expected output if you run just this (nothing calls anything yet): nothing — add `run_parallel()` and the two `asyncio.run(...)` calls to see the timings print.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
define:
    async def f1() -> str: await asyncio.sleep(2); return "f1 done"
    async def f2() -> str: await asyncio.sleep(2); return "f2 done"
    async def f3() -> str: await asyncio.sleep(2); return "f3 done"

define:
    async def run_sequential() -> None:
        start = time.perf_counter()
        await f1(); await f2(); await f3()
        print("sequential:", time.perf_counter() - start)

define:
    async def run_parallel() -> None:
        start = time.perf_counter()
        await asyncio.gather(f1(), f2(), f3())
        print("parallel:", time.perf_counter() - start)

if __name__ == "__main__":
    asyncio.run(run_sequential())
    asyncio.run(run_parallel())
```

Fill in `run_parallel` yourself, then compare both against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

make f2 raise an exception instead of succeeding, on purpose

run `gather(f1(), f2(), f3())` with no extra flag:

-> raises f2's exception, you never see f1 or f3's results at all

run `gather(f1(), f2(), f3(), return_exceptions=True)`:

-> returns a list of 3 things: f1's result, f2's exception object, f3's result

-> loop over the list, check `isinstance(item, Exception)` to tell results from

```
# gather_practice.py – Intermediate section
async def f2_broken() -> str:
    await asyncio.sleep(2)
    raise ValueError("f2 failed on purpose")

async def run_parallel_safe() -> None:
    results = await asyncio.gather(f1(), f2_broken(), f3(), return_exceptions=True)
    for result in results:
        if isinstance(result, Exception):
            print("a task failed:", result)
        else:
            print("a task succeeded:", result)
```

Wire this into a runnable script yourself — compare the output with and without

`return_exceptions=True` — then check the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate's pseudocode and near-complete code both assume every task succeeds. Advanced deliberately breaks one task and shows the two different outcomes `gather` can give you depending on `return_exceptions` — losing every result the moment one task fails (the default), versus getting every result back, successes and failures both labeled, in one list.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)